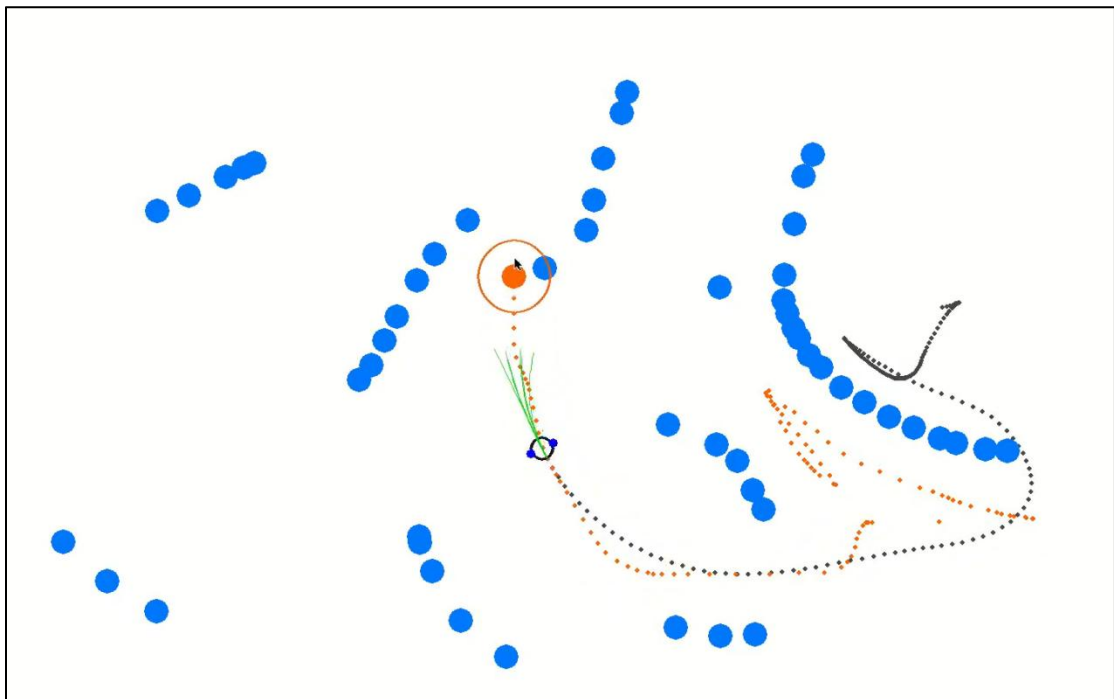


DWA Path Planning

1. DWA Algorithm

1.1 Description

The Dynamic Window Approach (DWA), a classic algorithm for path planning and motion control of mobile robots, ensures safe navigation on a known map. By exploring the speed and angular velocity state space, DWA identifies the optimal combination for safe navigation. Below, you'll find a basic description along with some key formulas of the DWA algorithm.



The core concept of the DWA algorithm involves the robot assessing its current state and sensor data to generate a series of potential motion trajectories (referred to as dynamic windows) in the speed and angular velocity state space. These trajectories are then evaluated based on criteria such as obstacle avoidance, maximizing forward speed, and minimizing angular velocity to select the optimal trajectory. Through iterative iterations of this process, the robot dynamically plans its trajectory in real-time to adapt to changing environments and obstacles.

1.2 Formula

1. Robot status: current position (x, y) and orientation (θ)
2. Motion control parameters: linear velocity (V) and angular velocity (ω).
3. Range of velocity and angular velocity sampling: minimum (V_{min} , ω_{min}) and maximum (V_{max} , ω_{max}).
4. Time step: Δt

The formula is as below:

A. **Velocity Sampling:** In the DWA algorithm, the initial step involves sampling the state space of velocity and angular velocity to create a set of potential velocity-angular velocity pairs, known as dynamic windows.

$$V_{samples} = [V_{min}, V_{max}]$$

$$\omega_{samples} = [-\omega_{max}, \omega_{max}]$$

($V_{samples}$) denotes the speed sampling range, while ($\omega_{samples}$) indicates the angular speed sampling range.

B. **Motion Simulation:** The DWA algorithm conducts a motion simulation for each speed-angular velocity pair, determining the trajectory of the robot based on these combinations in its current state.

$$x(t+1) = x(t) + v * \cos(\theta(t)) * \Delta t$$

$$y(t+1) = y(t) + v * \sin(\theta(t)) * \Delta t$$

$$\theta(t+1) = \theta(t) + \omega * \Delta t$$

Here, $x(t)$ and $y(t)$ denote the robot's position, $\theta(t)$ represents its orientation, v stands for linear velocity, ω for angular velocity, and Δt represents the time step.

C. **Trajectory Evaluation:** The DWA algorithm assesses each generated trajectory using evaluation functions, including obstacle avoidance, maximum speed, and minimum angular velocity.

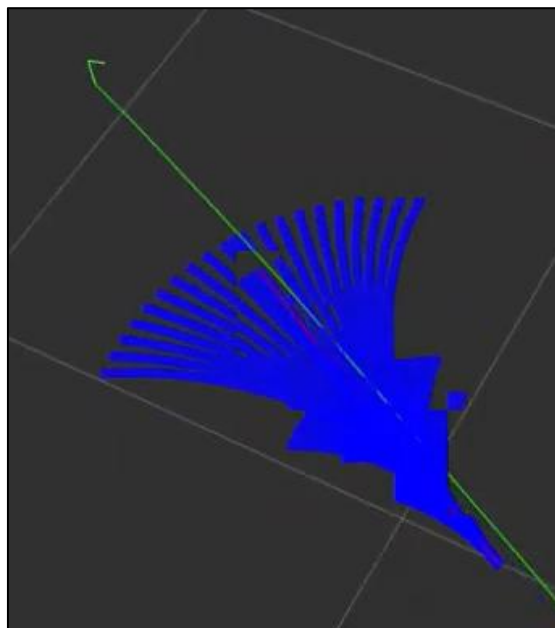
Obstacle Avoidance Evaluation: Detects if the trajectory intersects with obstacles.

Maximum Speed Evaluation: Verifies if the maximum linear speed on the trajectory falls within the permissible range.

Minimum Angular Velocity Evaluation: Ensures that the minimum angular velocity on the trajectory remains within the allowed range.

These evaluation functions can be defined and adjusted as per task requirements.

D. **Select optimal trajectory:** The DWA algorithm chooses the trajectory with the highest evaluation score as the next move for the robot.



1.3 Expansion

Extensions and resources for learning about the DWA algorithm:

The DWA algorithm serves as a fundamental method in mobile robotics, with numerous extended and enhanced versions designed to boost performance and efficiency in path planning. Some notable variations include:

1. DWA Algorithm Extension: <https://arxiv.org/abs/1703.08862>
2. Enhanced DWA (e-DWA) Algorithm: <https://arxiv.org/abs/1612.07470>)
3. DP-DWA Algorithm (DP-based Dynamic Window Approach):
<https://arxiv.org/abs/1909.05305>
4. ROS Wiki: http://wiki.ros.org/dwa_local_planner

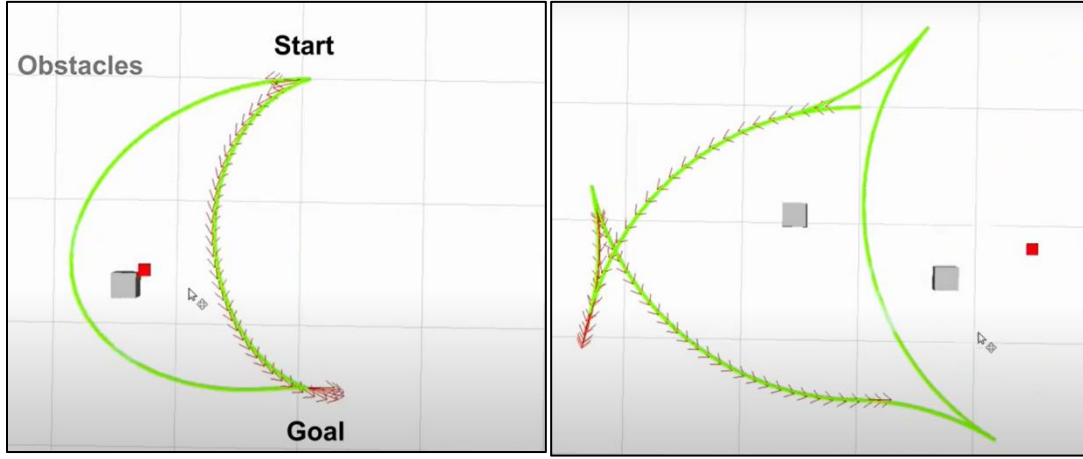
These resources provide in-depth insights and further exploration into the DWA algorithm and its various extensions.

2. TEB Algorithm

2.1 Description

The TEB (Timed Elastic Band) algorithm is utilized for both path planning and motion planning, particularly in domains like robotics and autonomous vehicles. At its core, the TEB algorithm treats path planning as an optimization challenge, aiming to generate the best trajectory within a specified timeframe while accommodating dynamic constraints and obstacle avoidance needs for the robot or vehicle. Key characteristics of the TEB algorithm encompass:

1. **Time Layered Representation:** The TEB algorithm employs time layering, dividing the trajectory into discrete time steps, each corresponding to a position of the robot or vehicle. This aids in setting timing constraints and preventing collisions.
2. **Trajectory Parameterization:** TEB parameterizes the trajectory into displacements and velocities, facilitating optimization. Each time step is associated with displacement and velocity parameters.
3. **Constrained Optimization:** TEB considers dynamic constraints, obstacle avoidance, and trajectory smoothness, integrating them into the objective function of the optimization problem.
4. **Optimization Solution:** TEB utilizes techniques like linear quadratic programming (QP) or nonlinear programming (NLP) to determine optimal trajectory parameters that fulfill the constraints.



2.2 Formula

The figure below illustrates the optimization objective function in the TEB algorithm:

$$J(x) = \sum_{i=1}^N \left[\omega_{\text{smooth}} * x_i^T * H * x_i + \omega_{\text{obstacle}} * \sum_{j=1}^M \int (x_i, o_j) \right]$$

In the following expression:

$J(x)$ denotes the objective function, where x represents the trajectory parameter.

ω_{smooth} and ω_{obstacle} represent weights assigned to smoothness and obstacle avoidance, respectively.

H signifies the smoothness penalty matrix.

$f(x_i, o_j)$ represents the obstacle cost function between trajectory point x_i and obstacle o_j .

1. Status Definition:

Firstly, we define the state of the robot (or vehicle) in the path planning problem as follows:

Position: $P = [X, Y]$, indicating the coordinates of the robot on the two-dimensional plane.

Velocity: $V = [V_x, V_y]$, representing the robot's velocity along the X and Y axes.

Time: t , denoting the current time.

Control Input: $u = [u_x, u_y]$, representing the control input of the robot, which

can be speed or acceleration.

Robot Trajectory: $x(t) = [p(t), v(t)]$, indicating the state of the robot at time t .

2. Target Function:

The essence of the TEB algorithm lies in solving an optimization problem. The objective is to minimize a composite function comprising various components:

$$J(x) = J_{\text{smooth}}(x) + J_{\text{obstacle}}(x) + J_{\text{dynamic}}(x)$$

$J_{\text{smooth}}(x)$: Smoothness objective function, ensuring trajectory smoothness.

$J_{\text{obstacle}}(x)$: Obstacle avoidance objective function, preventing collisions with obstacles.

$J_{\text{dynamic}}(x)$: Dynamic objective function, ensuring compliance with the robot's dynamic constraints.

3. Smoothness objective function $J_{\text{smooth}}(x)$:

Smoothness objective functions typically involve the curvature of trajectories to ensure the generated trajectories are smooth. It can be represented as:

$$J_{\text{smooth}}(x) = \int \kappa(t)^2 dt$$

Where $\kappa(t)$ is the curvature.

4. Obstacle avoidance objective function $J_{\text{obstacle}}(x)$:

The obstacle avoidance objective function calculates the distance between trajectory points and obstacles. It penalizes trajectory points that are in close proximity to obstacles. The specific obstacle cost function, denoted as $f(x, o)$, can be adjusted according to specific requirements or needs.

$$J_{\text{obstacle}}(x) = \sum_{o \in \text{obstacles}} \int f(x, o)$$

5. Dynamic objective function $J_{\text{dynamic}}(x)$:

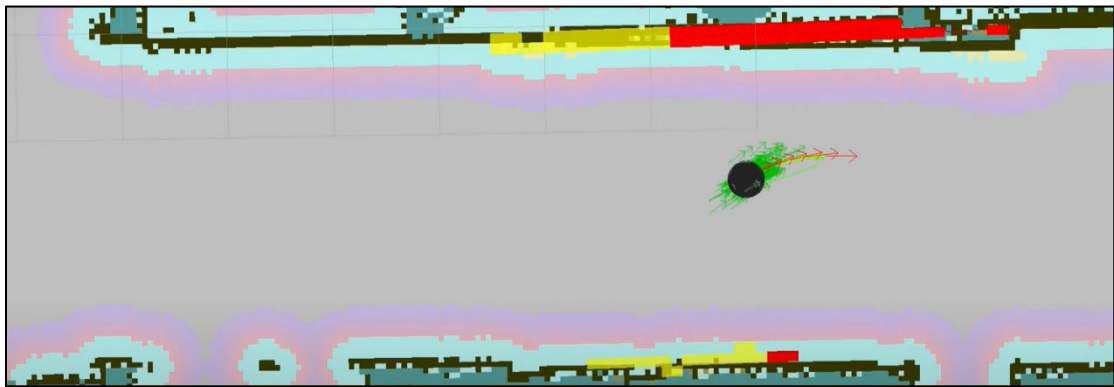
The dynamics objective function ensures that the generated trajectory adheres to the robot's dynamic constraints, which are determined by its kinematics and dynamics model. This typically includes limitations on velocity and

acceleration.

6. Optimization:

In the end, the Trajectory Optimization with Ellipsoidal Bounds (TEB) algorithm addresses the stated objective function by framing it as a constrained optimization problem. This problem incorporates optimization variables for trajectory parameters, time allocation, and control inputs. Typically, this optimization problem falls under the category of nonlinear programming (NLP) problems.

2.3 Expansion



Additional resources and learning materials for the TEB algorithm:

The TEB algorithm is a significant technology within the realm of path planning, boasting numerous extended and enhanced versions. Below are some learning links and extension topics aimed at facilitating a more comprehensive understanding of the TEB algorithm and its associated concepts:

1. Original TEB Paper: "**Trajectory modification considering dynamic constraints of autonomous robots**" by M. Rösmann et al.
2. TEB implementation in ROS: The TEB algorithm is commonly implemented as a ROS package (Robot Operating System Package), making it readily available for robot path planning tasks.

ROS TEB Local Planner Package:

https://github.com/rst-tu-dortmund/teb_local_planner

Wiki website:

http://wiki.ros.org/teb_local_planner

These links provide valuable resources for users seeking to explore the TEB algorithm and its associated topics in greater detail.